

Manual de Integracao e Seguranca - VisionCam V12.0

Manual de Integração, Segurança e Deploy - VisionCam V12.0

Este documento consolida toda a arquitetura, decisões técnicas, segurança de borda, gestão de custos de IA e o fluxo de implantação da solução **VisionCam** rodando de forma híbrida entre as placas **Radxa Cubie A7A** e a nuvem.

1. Desenho da Arquitetura de Contêineres (Radxa)

Na ponta (Edge), a placa Radxa operará com uma pilha de contêineres orquestrados via ``docker-compose.yml`` voltada à estabilidade, suporte remoto e baixa latência.

graph TD

ST

Componentes Locais:

- **Frigate NVR** (`^:5000``): Captura os fluxos RTSP, realiza detecção de movimento e pessoas com aceleração Rockchip NPU e grava clipes locais.

- **Mo**

2. Segurança e Gateway de Autenticação Local

Para proteger a integridade técnica da placa nas lojas físicas, o frontend local (port 3000) foi bloqueado por uma camada de segurança baseada em sessões.

- **Senha Inicial Padrão:** `admin` (armazenada em hash SHA256 no banco SQLite `system.db`).

- **En**

3. Frontend Técnico e Remoção do Paine

O lojista final **não acessa** o sistema local. A interface foi redesenhada com propósitos estritamente industriais e de suporte técnico:

1. **Remoção da Galeria de Auditoria** (`ui/app/audit`): Todo o histórico de vídeos, estatísticas de furtos e acompanhamento de auditorias foram deletados da interface local. Eles rodarão exclusivamente no painel centralizado em nuvem.

2. **D**
de CP

4. Gestão de Custos da API Gemini (Flash)

Para tornar a solução de monitoramento barata e viável comercialmente para pequenos mercados:

- **Conexão Indireta:** As placas Radxa não realizam chamadas diretas à API do Gemini de forma individual.

- **Cl**
do clip

5. Instalador de Primeiro Acesso (Bootstrap)

O script `bootstrap\_installer.py` foi projetado para atuar como o instalador "zero-dependency" do sistema. Ao iniciar uma placa Radxa limpa (contendo apenas o Python 3 e o Docker daemon):

1. O técnico executa ``python bootstrap_installer.py`` na placa.

2. Um

6. Fluxo de Publicação e Versionamento (Git & Docker)

Como as ferramentas Git e Docker rodam fora do ambiente Windows bare-metal, utilize os guias de comando abaixo nos seus terminais configurados:

A. Subindo o Código para o GitHub

O arquivo `.gitignore`` já está configurado para não subir caches de Python, binários YOLO (``*.pt``) e arquivos locais de banco de dados (``*.db``).

1. Inicializar Git localmente

git in

B. Gerando e Enviando as Imagens Docker

Foram criados os Dockerfiles do [backend core](file:///c:/Sistemas/Gemini/VisionCam/EdgeAI/Dockerfile) e do [frontend técnico ui](file:///c:/Sistemas/Gemini/VisionCam/EdgeAI/ui/Dockerfile) (com compilação Next.js de produção).

No terminal com suporte a Docker:

1. C

7. Métodos de Validação

- ****Validação de Autenticação:**** Rodando a suíte de testes `.\venv\Scripts\python scratch/test_v12_auth.py``, confirmando 5/5 testes passados (bloqueios de rotas, tokens inválidos e atualizações de senha).

- ****Va**
bootst

8. Bot de Administração Remota via Telegram

Para permitir a gestão, monitoramento e execução de comandos diretamente da ponta através do celular, implementamos um Bot de Administração Remota ([telegram_admin_bot.py](file:///c:/Sistemas/Gemini/VisionCam/EdgeAI/telegram_admin_bot.py)).

Funcionalidades:

- Ele roda em background de forma persistente monitorado pelo [entrypoint.py](file:///c:/Sistemas/Gemini/VisionCam/EdgeAI/entrypoint.py).

- ****Se**
tabela